Deep High-Order Tensor Convolutional Sparse Coding for Hyperspectral Image Classification

Chunbo Cheng, Hong Li, *Member, IEEE*, Jiangtao Peng, *Senior Member, IEEE*, Wenjing Cui, and Liming Zhang, *Member, IEEE*

***Abstract*— Most hyperspectral image (HSI) data exist in the form of tensor; the tensor representation preserves the potential spatial–spectral structure information compared with the vector representation, which can help improve the classification performance of HSI. In this article, a deep high-order tensor convolutional sparse coding (CSC) model is proposed, which can be used to train deep high-order filters. Based on the deep high-order tensor CSC model, a deep feature extraction network (DHTCSCNet) is constructed, which is used for feature extraction of HSIs. By combining the spectral–spatial feature and the features extracted by the proposed DHTCSCNet at each layer, a combined feature that incorporates shallow, deep, spectral, and spatial features can be obtained. Then, the graph-based learning (GSL) methods are used to classify the combined feature. Experimental results show that the DHTCSCNet can obtain better classification performance compared with other HSI classification methods.**

***Index Terms*— Deep high-order tensor convolutional sparse coding (CSC), deep learning, graph-based learning (GSL), hyperspectral image (HSI) classification.**

I. Introduction

HYPERSPECTRAL image (HSI) classification has been extensively studied in the field of remote sensing [1], [2]. However, because of large number of spectral channels, limited training samples, and large spatial variability, classification of hyperspectral data remains a challenging task [3], [4]. Therefore, how to extract fruitful features from such high-dimensional limited data efficiently and effectively has become a critical problem [2].

To this end, many approaches, such as dimensionality reduction [5] and band selection [6], had researched. These methods mainly focus on spectral feature extraction. Due to the complex spatial distribution and spectral heterogeneity of the objects [2], classification only relying on spectral features usually cannot obtain high classification accuracy [7]. Peng *et al.* [8] proposed a maximum likelihood estimation (MLE)-based JSR (MLEJSR) model for classification of HSI. Generally, using spatial–spectral features for classification can obtain better classification performance [9]. In order to simultaneously utilize the spectral and spatial features of HSIs, many scholars have proposed a variety of spectral–spatial-based classification methods [2], [10], [11]. Shen and Jia [12] designed a set of complex Gabor wavelets to extract the signal variance in the spatial, spectral, and joint spatial–spectral domains. The Markov random fields model [13] was introduced for HSI classification, which combined spatial and spectral features for classification [2]. In [14], the 2-D local binary pattern (LBP) had been extended into 3-D LBP (3-DLBP) model through forming a 3-D regular octahedral frame to characterize the spectral–spatial relationship. Li *et al.* [9] proposed a new framework for spectral–spatial classification of HSI data, which exploits both spatial and spectral information simultaneously. Gao *et al.* [15] proposed a spectral and spatial classification framework for HSI, which uses random multigraphs (RMGs). Liu *et al.* [16] presented sparse representation classification framework based on spatial–spectral kernel, which measures spatial similarity by neighborhood filtering of kernel feature space. Camps-Valls *et al.* [17] proposed a family of composite kernels based on Mercer's kernels, which combines spatial and spectral information. Li *et al.* [18] proposed a new framework called multiscale spatial information fusion (MSIF), which consists of three parts: multiscale spatial information extraction, local 1-D [19] embedding using spectral information and spatial information at different scales, and information fusion. However, these methods heavily depend on the quality of the hand-crafted features that are considered the shallow features.

Low-rank representation is one of the most effective feature extraction methods [20], [21], in which the principal component analysis (PCA) [20], nonnegative matrix

Manuscript received May 26, 2021; revised August 13, 2021 and November 16, 2021; accepted December 6, 2021. Date of publication December 10, 2021; date of current version March 1, 2022. This work was supported in part by the National Natural Science Foundation of China under Grant 61877021, Grant 42171351, and Grant 61871177; in part by the Natural Science Foundation of Hubei Province under Grant 2021CFA087; in part by the National Key Research and Development Program of China under Grant 2020YFA0714200; and in part by the Multi-year Research under Grant MYRG2018-00111-FST. *(Corresponding authors: Hong Li; Jiangtao Peng.)*

Chunbo Cheng and Wenjing Cui are with the School of Mathematics and Physics, Hubei Polytechnic University, Huangshi 435000, China (e-mail: bccheng@hbpu.edu.cn; wjcui@hbpu.edu.cn).

Hong Li is with the School of Mathematics and Statistics, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: hongli@hust.edu.cn).

Jiangtao Peng is with the Hubei Key Laboratory of Applied Mathematics, Faculty of Mathematics and Statistics, Hubei University, Wuhan 430062, China (e-mail: pengjt1982@hubu.edu.cn).

Liming Zhang is with the Faculty of Science and Technology, University of Macau, Macau 999078, China (e-mail: lmzhang@umac.mo).

Digital Object Identifier 10.1109/TGRS.2021.3134682

1558-0644 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:55:14 UTC from IEEE Xplore. Restrictions apply.

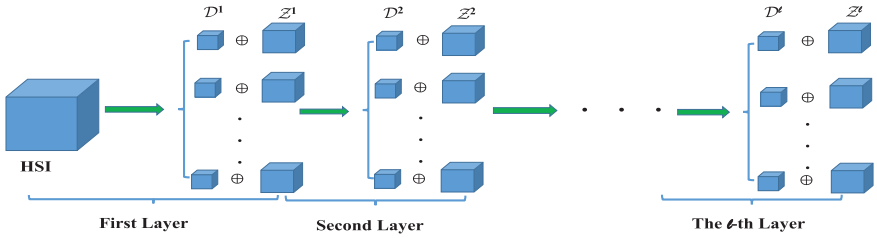


Fig. 1. Deep high-order tensor CSC for HSI. $\mathcal{D}^l$ represents the lth layer convolution kernel, and $\mathcal{Z}^l$ represents the lth layer sparse coding.

factorization (NMF) [22], and convolutional sparse coding (CSC)-based methods [23] are the most commonly used feature extraction methods. However, such methods need to transform the 3-D HSI into 2-D data, which will lose the spatial information contained in the HSI. It is obvious that the third-order tensor can represent HSI more naturally, and it has been widely used in hyperspectral classification [20], [24], [25] and hyperspectral unmixing [26]. However, feature extraction based on the tensor or matrix can only extract shallow features.

Recently, deep learning methods [2] have achieved great success in fields such as computer vision and image processing [27]–[29]. In HSI classification, a convolutional neural network (CNN) has shown great success. The reason why CNN has achieved such great success is mainly due to its ability to extract effective deep features that have stronger robustness and invariance [2] compared with shallow features. For CNN, its two important components include deep structure and convolution kernel. A deep structure can extract deep and abstract features, and convolutional architecture learns the spatial structure of images [20]. Unfortunately, these networks train the convolution kernels in an end-to-end manner, which is prone to overfitting. Therefore, many methods [20], [30] proposed to replace the process of learning these convolutional kernels with the traditional matrix or tensor decomposition method. For example, the unsupervised CNN model [31] learns convolutional kernels by employing CSC instead of backpropagation.

In this article, we propose a deep high-order tensor CSC model, as shown in Fig. 1, which can be used to train deep high-order filters. Based on the deep high-order tensor CSC model, a deep feature extraction network (DHTCSCNet) is constructed, which can be used to extract deep features. Combining the spectral and spatial features, and the features extracted by the proposed DHTCSCNet at each layer, and the learning model based on the graph is used for classification. The DHTCSCNet has higher classification accuracy than other HSI classification methods.

HSI classification methods based on deep learning can learn more complex, robust, and powerful deep feature representations than conventional HSI classification methods. The powerful deep feature representation capability of deep learning methods is very helpful for HSI classification.

Recently, HSI classification methods based on deep learning almost all use CNN. Among these HSI classification methods based on CNN, the initial weight of the convolutional neural network is generated by random initialization. The selection of the initial weights of the HSI classification methods based on CNN is very critical. In some methods, the pretraining strategy is used to train the initial weights. The advantages of this pretraining strategy are given as follows: 1) a good selection of initial weights can greatly reduce the time of network training and 2) it can not only initialize the weights near a basin of attraction of local minimum that produces better generalization performance but also restrict the initialized weights inside a basin of attraction to a good local minimum without significantly changing the network weights [32]. Therefore, for CNNs, we aim to propose a pretraining strategy to improve the accuracy of HSI classification.

In CNNs, the network weight is equivalent to the filter, and the training purpose of the network is to obtain these general filters, while high-order tensor CSC can obtain the filters in an unsupervised manner, which performs multilayer convolutional sparse decomposition of HSI data in the tensor form and generates a set of highly diverse filters. However, one of the disadvantages of high-order tensor CSC is that it can only extract shallow features and cannot learn deep filters. Therefore, we aim to propose a deep high-order tensor CSC that can extract deep features and learn deep filters from HSI. Thus, the initial values of parameters in CNNs can be trained by our deep high-order tensor CSC model.

The main contributions and novelty of this article are summarized as follows.

1) A deep high-order tensor CSC model is proposed, which can be used to train deep high-order filters, while conventional high-order tensor CSC models can only learn shallow features and shallow filters.
2) Based on the deep high-order tensor CSC model, a DHTCSCNet is constructed, which is used for feature extraction of HSIs. The proposed DHTCSCNet combines shallow, deep, spectral, and spatial features in the classification.
3) A novel pretraining strategy is proposed. Specifically, the filters trained by the deep high-order tensor CSC model are taken as the initial value of the DHTCSCNet, and the DHTCSCNet has no backpropagation and fine-tuning process.

The remainder of this article is organized as follows. Section II introduces the preliminary knowledge of high-order tensor CSC. Section III presents the proposed deep tensor CSC model, DHTCSCNet, for feature extraction and the classification framework. Sections IV and V are experiments and conclusion, respectively.

II. Preliminary

CSC has achieved great success as image denoising and a classification tool. High-order tensor CSC was first proposed

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:55:14 UTC from IEEE Xplore. Restrictions apply.

by [33]. The arbitrary dth-order tensor CSC problem [33] can be expressed as

$$\min_{\mathcal{D},\mathcal{Z}_n} \frac{1}{2}\sum_{n}^{N} \|\mathcal{I}_n - \mathcal{D} \circledast_{\mathrm{HT}} \mathcal{Z}_n\|_F^2 + \lambda \|\mathcal{Z}_n\|_{\underbrace{1,\ldots,1}_{d+1}}$$
$$\text{s.t. } \|\mathcal{D}_m\|_F^2 \le 1 \quad \forall m = 1,\ldots,M \tag{1}$$

where $\circledast_{\mathrm{HT}}$ represents a higher order tensor convolution operator, which is defined in the literature [33]. The nth multidimensional training image is $\mathcal{I}_n \in \mathbb{R}^{n_1\times 1\times n_2\times\cdots\times n_d}$. The filter $\mathcal{D}_m \in \mathbb{R}^{n_1\times 1\times n_2\times\cdots\times n_d}$ and the M filters $\{\mathcal{D}_m\}_{m=1}^{M}$ are concatenated to the dictionary $\mathcal{D} \in \mathbb{R}^{n_1\times M\times n_2\times\cdots\times n_d}$, and the nth sparse code is $\mathcal{Z}_n \in \mathbb{R}^{M\times 1\times n_2\times\cdots\times n_d}$. For an Pth-order tensor $\mathcal{I} \in \mathbb{R}^{J_1\times J_2\times\cdots\times J_P}$, the Frobenius squared norm is $\|\mathcal{I}\|_F^2 = \sum_{i_1,i_2,\ldots,i_P} \mathcal{I}(i_1,i_2,\ldots,i_P)^2$, and $\ell_{1,\ldots,1}$ norm of $\mathcal{I}$ is $\|\mathcal{I}\|_{1,\ldots,1} = \sum_{i_1,i_2,\ldots,i_P} |\mathcal{I}(i_1,i_2,\ldots,i_P)|$, where $i_k = 1,\ldots,J_k$ $(k = 1,\ldots,P)$, and $J_k \in \mathbb{Z}^+$ (positive integer) represents the size of the kth mode (i.e., it contains the number of dimensions).

The traditional fixed point strategy is used to solve the high-order tensor CSC in (1), where $\mathcal{D}$ and $\mathcal{Z}_n$ can be solved alternately. Each subproblem is convex, so the alternating direction method of multipliers (ADMM) can be used to solve it.

III. Deep High-Order Tensor CSC for Hyperspectral Image Classification

In this section, we will introduce a deep high-order tensor CSC (DHTCSC) model for HSI classification.

A. Deep High-Order Tensor CSC Model

In this section, we build a DHTCSC model, which can extract deep features efficiently.

Given a training set including N HSI data, the deep fourth-order tensors CSC model can be used to decompose HSI in multiple layers to obtain the convolutional kernels, in which N represents the number of HSIs. It can easily be stacked to form a hierarchy by treating the feature maps of layer l as input for layer $l+1$. In other words, layer l has as its input an image with K_{l-1} feature maps at layer $l-1$, as shown in Fig. 1. The convolutional kernels of layer l can be obtained by minimizing the cost function $\mathcal{C}_l$ at the lth layer of the fourth-order tensors' CSC model

$$\min_{\mathcal{D}^l,\mathcal{Z}^l} \frac{1}{2}\sum_{n=1}^{N}\sum_{c=1}^{K_{l-1}}\left\|\sum_{k=1}^{K_l}\mathcal{D}_{k,c}^{l} \circledast_{\mathrm{HT}} \mathcal{Z}_{k,l}^{n} - \mathcal{Z}_{c,l-1}^{n}\right\|_F^2 + \lambda\sum_{n=1}^{N}\sum_{k=1}^{K_l}\left\|\mathcal{Z}_{k,l}^{n}\right\|_{1,1,1,1,1}$$
$$\text{s.t. } \left\|\mathcal{D}_{k,c}^{l}\right\|_F^2 \le 1 \quad \forall k = 1,\ldots,K_l,\ c = 1,\ldots,K_{l-1} \tag{2}$$

where $\circledast_{\mathrm{HT}}$ represents a fourth-order tensor convolution operator. $\mathcal{Z}_{k,l}^{n}$ represents the sparse coding of the kth channel of the nth HSI in the lth layer. $\mathcal{D}_{k,c}^{l}$ is the cth dimension of the kth convolutional kernel in the lth layer. The penalty parameter $\lambda > 0$ controls the tradeoff between good reconstruction and code sparsity. For a fourth-order tensor $\mathcal{I} \in \mathbb{R}^{J_1\times J_2\times J_3\times J_4}$, $\|\mathcal{I}\|_F^2 = \sum_{i_1,i_2,i_3,i_4}\mathcal{I}(i_1,i_2,i_3,i_4)^2$, and the $\ell_{1,1,1,1,1}$ norm is $\|\mathcal{Z}\|_{1,1,1,1,1} = \sum_{i_1,i_2,i_3,i_4}|\mathcal{Z}(i_1,i_2,i_3,i_4)|$. It should be noted here that, when l is equal to 1, $\mathcal{Z}_{c,0}^{n} = \mathcal{I}_n \in \mathbb{R}^{n_1\times 1\times n_2\times n_3\times n_4}$.

The deep fourth-order tensors CSC model is trained in a bottom-up manner, the feature maps, and convolutional kernels at the layer l obtained by minimizing the cost function $\mathcal{C}_l$. For each layer, (2) is a high-order tensor CSC problem in Section II, which can be divided into two subproblems to be solved. In this way, we can solve the sparse coding $\mathcal{Z}^l$ and the convolutional kernel $\mathcal{D}^l$ of each layer, respectively. Specifically, we use ADMM to solve the sparse coding $\mathcal{Z}^l$ and the gradient descent algorithm to solve the convolutional kernel $\mathcal{D}^l$, and the specific solution process is given as follows.

1) Sparse Coding: Fixing $\mathcal{D}^l$, we solve $\mathcal{Z}^l$ in (2). The augmented lagrangian is given as follows:

$$\begin{aligned}\mathcal{L}(\mathcal{Z}^l,\mathcal{Y}^l,\mathcal{U}^l) &= \frac{1}{2}\sum_{n=1}^{N}\sum_{c=1}^{K_{l-1}}\left\|\sum_{k=1}^{K_l}\mathcal{D}_{k,c}^{l} \circledast_{\mathrm{HT}} \mathcal{Z}_{k,l}^{n} - \mathcal{Z}_{c,l-1}^{n}\right\|_F^2 \\ &\quad + \frac{\rho}{2}\sum_{n=1}^{N}\sum_{k=1}^{K_l}\left\|\mathcal{Z}_{k,l}^{n} - \mathcal{Y}_{k,l}^{n}\right\|_F^2 + \lambda\sum_{n=1}^{N}\sum_{k=1}^{K_l}\left\|\mathcal{Y}_{k,l}^{n}\right\|_{1,1,1,1,1} \\ &\quad + \sum_{n=1}^{N}\sum_{k=1}^{K_l}\left\langle \mathcal{U}_{k,l}^{n}, \left(\mathcal{Z}_{k,l}^{n} - \mathcal{Y}_{k,l}^{n}\right)\right\rangle\end{aligned} \tag{3}$$

where $\mathcal{Y}^l$ and $\mathcal{U}^l$ represent the primal variable and the dual variable at the lth layer, respectively. ρ is a continuation parameter. $\mathcal{Y}_{k,l}^{n}$ represents the primal variable of the kth channel of the nth HSI in the lth layer. $\mathcal{U}_{k,l}^{n}$ represents the dual variable of the kth channel of the nth HSI in the lth layer.

2) Update $\mathcal{Z}^l$: For computational efficiency, we can use the definition of $\circledast_{\mathrm{HT}}$ to transform the objective (3) in the Fourier domain. It can be seen from (3) that the solution for each $\mathcal{Z}_{k,l}^{n}$ is independent of other variables. Here, we take derivatives of $\mathcal{L}(\hat{\mathcal{Z}}^l,\hat{\mathcal{Y}}^l,\hat{\mathcal{U}}^l)$ (Fourier domain) with respect to $\hat{\mathcal{Z}}_{k,l}^{n}$

$$\frac{\partial\mathcal{L}(\hat{\mathcal{Z}}^l,\hat{\mathcal{Y}}^l,\hat{\mathcal{U}}^l)}{\partial\hat{\mathcal{Z}}_{k,l}^{n}} = \sum_{c=1}^{K_{l-1}}\left(\hat{\mathcal{D}}_{k,c}^{l}\right)^H\left(\sum_{k=1}^{K_l}\hat{\mathcal{D}}_{k,c}^{l}\hat{\mathcal{Z}}_{k,l}^{n} - \hat{\mathcal{Z}}_{c,l-1}^{n}\right) + \rho\left(\hat{\mathcal{Z}}_{k,l}^{n} - \hat{\mathcal{Y}}_{k,l}^{n}\right) + \hat{\mathcal{U}}_{k,l}^{n}. \tag{4}$$

For a fixed n, setting $((\partial\mathcal{L}(\hat{\mathcal{Z}}^l,\hat{\mathcal{Y}}^l,\hat{\mathcal{U}}^l))/\partial\hat{\mathcal{Z}}_{k,l}^{n}) = 0$, for any given k, the optimal $\hat{\mathcal{Z}}_{k,l}^{n}$ is the solution to the following linear system:

$$\tilde{\mathcal{D}}\begin{pmatrix}\hat{\mathcal{Z}}_{1,l}^{n}\\ \vdots\\ \hat{\mathcal{Z}}_{K_l,l}^{n}\end{pmatrix} = \begin{pmatrix}\sum_{c=1}^{K_{l-1}}\left(\hat{\mathcal{D}}_{1,c}^{l}\right)^H\hat{\mathcal{Z}}_{c,l-1}^{n} + \rho\hat{\mathcal{Y}}_{1,l}^{n} - \hat{\mathcal{U}}_{1,l}^{n}\\ \vdots\\ \sum_{c=1}^{K_{l-1}}\left(\hat{\mathcal{D}}_{K_l,c}^{l}\right)^H\hat{\mathcal{Z}}_{c,l-1}^{n} + \rho\hat{\mathcal{Y}}_{K_l,l}^{n} - \hat{\mathcal{U}}_{K_l,l}^{n}\end{pmatrix} \tag{5}$$

where

$$\tilde{\mathcal{D}} = \begin{pmatrix}\mathcal{D}_{11} + \rho\boldsymbol{E} & \cdots & \mathcal{D}_{1K_l}\\ \vdots & \vdots & \vdots\\ \mathcal{D}_{K_l1} & \cdots & \mathcal{D}_{K_lK_l} + \rho\boldsymbol{E},\end{pmatrix} \tag{6}$$

$$\mathcal{D}_{nm} = \sum_{c=1}^{K_{l-1}}\left(\hat{\mathcal{D}}_{n,c}^{l}\right)^H\hat{\mathcal{D}}_{m,c}^{l}, \quad n,\ m = 1,\ldots,K_l. \tag{7}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:55:14 UTC from IEEE Xplore. Restrictions apply.

$(\cdot)^H$ is the conjugate transpose. Equation (5) can be effectively minimized by conjugate gradient (CG) descent. Of course, $\mathcal{Z}^l$ can be reconstructed back from $\hat{\mathcal{Z}}^l$ by taking the inverse Fourier transform $\mathcal{Z}^l = \mathcal{F}^{-1}(\hat{\mathcal{Z}}^l)$, where $\mathcal{F}^{-1}(\cdot)$ is denoted as the inverse Fourier transform.

3) Update $\hat{\mathcal{Y}}^l$:

$$\hat{\mathcal{Y}}_{k,l}^n \longleftarrow \underset{\hat{\mathcal{Y}}_{k,l}^n}{\operatorname{argmin}} \lambda \sum_{k=1}^{K_l} \left\|\hat{\mathcal{Y}}_{k,l}^n\right\|_{1,1,1,1,1} + \frac{\rho}{2} \sum_{k=1}^{K_l} \left\| \hat{\mathcal{Y}}_{k,l}^n - \overbrace{\left(\hat{\mathcal{Z}}_{k,l}^n + \frac{1}{\rho}\hat{\mathcal{U}}_{k,l}^n\right)}^{\mathcal{B}_n^l} \right\|_F^2, \quad n = 1, \ldots, N. \tag{8}$$

$\mathcal{S}_{(\lambda/\xi)}(\alpha) = \operatorname{sign}(\alpha)\max(0, |\alpha| - (\lambda/\xi))$ is used to solve $\mathcal{Y}_{k,l}^n (\forall n = 1, \ldots, N)$. It operates on the tensor $\mathcal{B}_n^l$ element by element.

4) Update $\hat{\mathcal{U}}^l$:

$$\hat{\mathcal{U}}_{k,l}^n \longleftarrow \hat{\mathcal{U}}_{k,l}^n + \rho\left(\hat{\mathcal{Z}}_{k,l}^n - \hat{\mathcal{Y}}_{k,l}^n\right). \tag{9}$$

5) Convolutional Kernel Learning: Fixing $\mathcal{Z}_{k,l}^n$ and n, we used the gradient descent method to update $\mathcal{D}^l$ by using the diagonalization property of $\circledast_{\mathrm{HT}}$

$$\frac{\mathcal{L}\left(\mathcal{Z}^l, \mathcal{Y}^l, \mathcal{U}^l\right)}{\partial \mathcal{D}_{k,c}^l} = \sum_{n=1}^{N} \sum_{c=1}^{K_{l-1}} \left(\mathcal{Z}_{k,l}^n\right)^T \left(\sum_{k=1}^{K_l} \mathcal{Z}_{k,l}^n \mathcal{D}_{k,c}^l - \mathcal{Z}_{c,l-1}^n \right). \tag{10}$$

Algorithm 1 summarizes the process of training convolutional kernel with the proposed method.

Algorithm 1 DHTCSC Model

Input: N HSI data $\{\mathcal{I}_n\}_{n=1}^N$, number of feature maps K;
Output: Convolutional kernel $\mathcal{D}$

1: Initialize each layer parameters: feature maps $\mathcal{Z}_{k,l}^n \sim \mathcal{N}(0, \epsilon)$; convolutional kernel $\mathcal{D}_{k,c}^l \sim \mathcal{N}(0, \epsilon)$; $\rho = \alpha = 0.01$; regularization parameter: $\nu = 1$; $\mathcal{Y}_{k,l}^n = \mathcal{U}_{k,l}^n = 0$.
2: Precompute: fast Fourier transforms (FFTs) of $\{\mathcal{I}_n\}_{n=1}^N$ and $\mathcal{Z}_{k,l}^n$.
3: **for** epoch=1:E **do**
4: **for** n=1:N **do**
5: Compute FFTs of $\mathcal{D}_{k,c}^l$, $\mathcal{Y}_{k,l}^n$ and $\mathcal{U}_{k,l}^n$;
6: $\forall\, k$, given $\hat{\mathcal{D}}_{k,c}^l$ and $\hat{\mathcal{Y}}_{k,l}^n$, solve for $\hat{\mathcal{Z}}_{k,l}^n$ using (5);
7: $\forall\, k$, given $\hat{\mathcal{D}}_{k,c}^l$ and $\hat{\mathcal{Z}}_{k,l}^n$, solve for $\hat{\mathcal{Y}}_{k,l}^n$ using (8);
8: $\forall\, k$, update Lagrange multiplier $\hat{\mathcal{U}}_{k,l}^n$ using (9);
9: $\forall\, k$, Compute inverse FFTs of $\hat{\mathcal{Z}}_{k,l}^n$;
10: **end for**
11: $\forall\, k, c$, update $\mathcal{D}_{k,c}^l$ using gradient descent in (10).
12: **end for**
13: **Return:** Convolutional kernel $\mathcal{D}^l$

B. Feature Extraction for HSI

Based on the deep high-order tensor CSC model, we construct a DHTCSCNet, as shown in Fig. 2. Different from the traditional deep learning methods, the traditional deep learning methods need to first perform pretraining and then fine-tune the parameters when performing feature extraction, while DHTCSCNet does not have a fine-tuning process in the feature extraction stage. Instead, by regarding the learned convolutional kernel as the initial weight of the convolutional neural network, DHTCSCNet can extract deep features efficiently without backpropagation and fine-tuning process.

1) Spectral–Spatial Feature Extraction: $\boldsymbol{H} \in \mathbb{R}^{r\times c\times m}$ is the original HSI data, and $r \times c \times m$ represents the size of HSI. In order to extract the spatial–spectral features of HSI, we use PCA and LBP for feature extraction. Specifically, the first t principal components are retained, and each principal component is locally binary coded. For the pixel centered at $\boldsymbol{P}_c$ and image patch of size $d \times d$, binary coding for each pixel on image patch can be given by

$$\sum_{j=0}^{d^2-1} \mathcal{S}\left(\boldsymbol{P}_j - \boldsymbol{P}_c\right) 2^j \tag{11}$$

where, if $\boldsymbol{P}_j (j = 0, 1, \ldots, d^2 - 1)$ is greater than $\boldsymbol{P}_c$, then $\mathcal{S}(\boldsymbol{P}_j - \boldsymbol{P}_c) = 1$; otherwise, $\mathcal{S}(\boldsymbol{P}_j - \boldsymbol{P}_c) = 0$.

As the combination of spectral information and spatial information can enhance the classification performance, spectral and spatial features $\tilde{\boldsymbol{H}} \in \mathbb{R}^{r\times c\times(m+t)}$ can be obtain by combining $\boldsymbol{H}$ and LBP feature.

2) Deep Feature Extraction of the DHTCSCNet: $\tilde{\boldsymbol{H}}$ is used as the input of CNN; we can build a deep structure network (called DHTCSCNet) to extract deep features; and the network structure is shown in Fig. 2. For each layer in DHTCSCNet, it contains the same convolution and ReLu (i.e., rectified linear units) operation. For deep networks, DHTCSCNet, the selection of initial values is very important. Here, we use the convolution kernels trained in the previous section as the initial weight of DHTCSCNet. Specifically, the deep higher order tensor CSC model in the previous section is used to train the weights of each layer for DHTCSCNet. Thus, the feature $\boldsymbol{F}^l$ of any layer l can be obtained, which can be calculated by the following formula:

$$\begin{aligned} \boldsymbol{F}_i^l &= \sum_{j=1}^{p} \boldsymbol{F}_j^{l-1} \oplus \boldsymbol{D}_{ij}^{l-1}, \quad i = 1, 2, \ldots, p \\ \boldsymbol{F}^l &= \max\left(0, \boldsymbol{F}^l - \boldsymbol{M}^l\right) \end{aligned} \tag{12}$$

where $\oplus$ denotes the 2-D convolution operator. $\boldsymbol{D}^{l-1}$ represents the $l-1$ layer convolutional kernel learned in section III-A, and $\boldsymbol{D}_{ij}^{l-1} \in \mathbb{R}^{n_1 n_2 n_3 \times p}$ is the jth dimension of the ith convolutional kernel in the $(l-1)$th layer. $\boldsymbol{M}^l \in \mathbb{R}^{rc\times p}$ is the mean matrix of $\boldsymbol{F}^l \in \mathbb{R}^{rc\times p}$. $\boldsymbol{F}_i^{l-1} \in \mathbb{R}^{rc\times 1}$ is the ith feature map in the $(l-1)$th layer, and $\boldsymbol{F}_i^l \in \mathbb{R}^{rc\times 1}$ is feature map; here, i and l represent the feature map index and the layer index, respectively.

C. HSI Classification

Combining the features of all layers $\boldsymbol{F}^1, \boldsymbol{F}^2, \ldots, \boldsymbol{F}^l, \boldsymbol{H}$, and $\tilde{\boldsymbol{H}}$, we can get the combined feature $\mathcal{F}_{\mathrm{fus}}$. For each dimension of $\mathcal{F}_{\mathrm{fus}}$, it can be standardized as

$$\hat{\mathcal{F}}_{\mathrm{fus}}^i = \frac{\mathcal{F}_{\mathrm{fus}}^i - \operatorname{mean}\left(\mathcal{F}_{\mathrm{fus}}^i\right)}{\operatorname{var}\left(\mathcal{F}_{\mathrm{fus}}^i\right)}, \quad i \in \{1, 2, \ldots, lp + 2m + t\} \tag{13}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:55:14 UTC from IEEE Xplore. Restrictions apply.

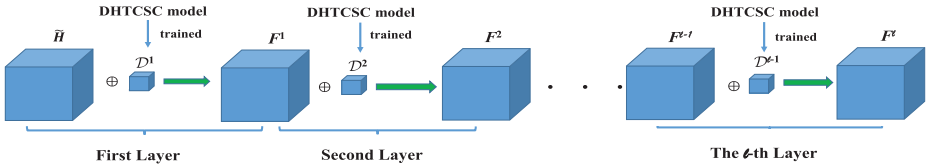


Fig. 2. Deep feature extraction of the DHTCSCNet. $\boldsymbol{F}^l$ represents the lth layer feature map, and $\mathcal{D}^l$ represents the lth layer convolution kernel.

where $\hat{\mathcal{F}}_{\text{fus}}^j \in \mathbb{R}^{rc\times 1}$ denotes the normalized data, and mean$(\cdot)$ and var$(\cdot)$ represent the mean and variance, respectively. By concatenating all $\hat{\mathcal{F}}_{\text{fus}}^j$, the feature $\hat{\mathcal{F}}_{\text{fus}} \in \mathbb{R}^{rc\times(lp+2m+t)}$ is obtained, and we use the graph-based learning (GSL) method to classify $\hat{\mathcal{F}}_{\text{fus}}$.

For the GSL method [2], we need to divide $\hat{\mathcal{F}}_{\text{fus}}$ into training samples $\mathcal{Y}_{lab} = [\boldsymbol{y}_1, \boldsymbol{y}_2, \ldots, \boldsymbol{y}_l]$ and test samples $\mathcal{Y}_{unlab} = [\boldsymbol{y}_{l+1}, \boldsymbol{y}_{l+2}, \ldots, \boldsymbol{y}_N]$. For hyperspectral data with k-class, the GSL is used to classify, which can be solved by the following quadratic optimization problem:

$$\min_{\boldsymbol{T}} \text{tr}(\boldsymbol{T}-\boldsymbol{V})^T\boldsymbol{M}(\boldsymbol{T}-\boldsymbol{V}) + \text{tr}(\boldsymbol{T}^T\boldsymbol{G}\boldsymbol{T}) \tag{14}$$

where the trace function is represented by tr$(\cdot)$ and $\boldsymbol{T} \in \mathbb{R}^{N\times k}$ represents the label to be predicted

$$\boldsymbol{V} = \begin{pmatrix} v_{11} & \cdots & v_{1k} \\ \vdots & \ddots & \vdots \\ v_{N1} & \cdots & v_{Nk} \end{pmatrix}, \quad \begin{cases} v_{ij} = 1, & \text{if } \boldsymbol{y}_i \in j \\ v_{ij} = 0, & \text{otherwise.} \end{cases} \tag{15}$$

$\boldsymbol{M} \in \mathbb{R}^{N\times N}$; it is defined as follows:

$$\boldsymbol{M} = \begin{pmatrix} m_1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & m_N \end{pmatrix}, \quad \begin{cases} m_i = \delta_n, & \text{if } 1 \le i \le n \\ m_i = \delta_{N-n}, & \text{otherwise} \end{cases} \tag{16}$$

where δ_n and δ_{N-n} are two preset parameters. The graph Laplacian $\boldsymbol{G} \in \mathbb{R}^{N\times N}$ is calculated by

$$\boldsymbol{G} = \boldsymbol{L} - \boldsymbol{W} \tag{17}$$

where $\boldsymbol{W} = \{\omega_{ij}\}(i, j = 1, \ldots, N) \in \mathbb{R}^{N\times N}$, and ω_{ij} is defined as follows:

$$\omega_{ij} = e^{-\frac{\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2}{2\vartheta^2}} (i \ne j), \quad \omega_{ii} = 0 \tag{18}$$

where ϑ is the preset parameter and $\boldsymbol{L}$ is defined by

$$\boldsymbol{L} = \begin{pmatrix} d_1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & d_N \end{pmatrix}, \quad d_i = \sum_{j=1}^{N} \omega_{ij}. \tag{19}$$

Algorithm 2 summarizes the whole framework of HSI classification.

D. Complexity Analysis

The DHTCSCNet is divided into two stages: the training stage and the testing stage. Specifically, in the training stage, the convolution kernel $\mathcal{D}^l \in \mathbb{R}^{n_1\times p\times n_2\times n_3}$ of each layer is learned; in the testing stage, the learned convolution kernel $\mathcal{D}^l$ is used for feature extraction, and the extracted features

Algorithm 2 HSI Classification Framework

Input: The original HSI $\boldsymbol{H}$; the convolutional kernel $\mathcal{D}^l$.
Output: Label $\boldsymbol{T}$.

1: **for** Layer i=1:l **do**
2: Initialize $t = 3$;
3: Applying PCA and LBP to extract spectral, spatial feature, and combine $\tilde{\boldsymbol{H}}$;
4: Using (12) to obtain the feature $\boldsymbol{F}^l$ of each layer.
5: **end for**
6: Combining the the features of all layers $\boldsymbol{F}^1, \boldsymbol{F}^2, \ldots, \boldsymbol{F}^l$, $\boldsymbol{H}$ and $\tilde{\boldsymbol{H}}$ to obtain $\mathcal{F}_{\text{fus}}$,
7: Normalize the feature $\mathcal{F}_{\text{fus}}$ to obtain $\hat{\mathcal{F}}_{\text{fus}}$.
8: Classify the feature $\hat{\mathcal{F}}_{\text{fus}}$ to obtain the class label $\boldsymbol{T}$.

are used for classification. Next, we will evaluate the computational complexity of these two stages, respectively.

For the training stage, the DHTCSCNet needs to train the convolution kernel and feature map of each layer. For the sparse coding step, the most expensive part is solving for $\mathcal{Z}^l$, which involves taking n_1 2-D Fourier transforms of size $n_2 \times n_3$ and solving $n_2 n_3$ linear systems each of size $p \times p$. Therefore, the total cost of updating the sparse codes can be estimated to be $\mathcal{O}_s = \mathcal{O}(n_2 n_3 p^3) + \mathcal{O}(n_1 n_2 n_3 log(n_2 n_3))$. Updating convolution kernel $\mathcal{D}^l$, the main computational cost comes from calculating the gradient $((\partial\mathcal{L}(\mathcal{D}^l, \mathcal{H}^l, \mathcal{V}^l))/(\partial\mathcal{D}_{k,c}^l))$ in (10), and its computational complexity is $\mathcal{O}_c = \mathcal{O}(N(K_l W_{l-1} H_{l-1} p n_1 n_2 n_3 + K_{l-1} W_l H_l p n_1 n_2 n_3))$, in which W_l and H_l represent the width and height of the lth layer feature map, respectively. Therefore, we obtain the total computational complexity of the lth layer: $\mathcal{O}_s + \mathcal{O}_c$.

For the testing stage, the DHTCSCNet has no backpropagation and fine-tuning process, which is used to extract features. In the feature extraction stage, it includes two processes: the spectral–spatial feature extraction and deep feature extraction. For the spectral–spatial feature extraction, we use PCA and LBP. The computational complexity of PCA and LBP is given as follows: 1) for the HSI $\boldsymbol{H} \in \mathbb{R}^{r\times c\times m}$, the computational complexity of PCA is $\mathcal{O}(\text{rcm}^2 + m^3 + \text{rcmt})$ and 2) for LBP, computational complexity of LBP is $\mathcal{O}(\text{rctd}^2 - \text{rct})$.

For the deep feature extraction, each layer contains the convolution operation and ReLU, and their total computational complexity is $\mathcal{O}(\text{ptrcd}^2 + \text{prct}) + \mathcal{O}(3\text{rcp})$. Assuming that $t \ll m \ll \text{rc}$, the computational complexity of DHTCSCNet with $l \ge 2$ layers can be approximated as $\mathcal{O}(\text{rcm}^2 + (l-1)\text{rcp}^2 + \text{lptrcd}^2)$. The computational complexity of the testing stage is listed in Table I in detail. As can be seen from the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:55:14 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPLEXITY FOR THE TESTING STAGE

Methods	Computational complexity
PCA	$\mathcal{O}(rcm^2 + m^3 + rcmt)$
LBP	$\mathcal{O}(rctd^2 - rct)$
Convolution	$\mathcal{O}(ptrcd^2 + prct)$
ReLU	$\mathcal{O}(3rcp)$
DHTCSCNet with l layers	$\mathcal{O}(rcm^2 + (l-1)rcp^2 + lptrcd^2)$

table, PCA and convolution operations are the most time-consuming in the testing stage.

Therefore, the total computational complexity of the DHTCSCNet includes the complexity of the training and testing phases. In fact, once the training phase of the convolution kernel is completed, the deep features can be directly extracted without the backpropagation process and fine-tuning during the test phase; this can greatly reduce the computational cost. In subsequent experiments, we numerically measured the running time.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

This section will conduct experiments on the selected datasets to verify the effectiveness of the proposed method.

A. Datasets Description and Parameter Settings

1) Datasets Description:

1) Pavia University dataset (PaviaU)[1]: It was obtained by the reflection optical system imaging spectrometer sensor, which contains nine classes, 610×340 pixels, and 103 spectral bands.
2) Indian Pines dataset (IP)[2]: It was obtained by the Airborne Visible/Infrared Imaging Spectrometer (AVIRIS) sensor [34]. The size of the IP dataset with 16 classes is 145×145.
3) KSC dataset[2]: It was collected by the AVIRIS sensor [34], which contains 512×614 pixels, 176 bands, and 13 classes.

According to the literature [2], [4], the number of training samples and testing samples on the three datasets can be determined, as shown in Tables II–IV, respectively.

2) Parameter Settings: Here, we use overall accuracy (OA) to measure the classification performance

$$\mathbf{OA} = \frac{1}{Q}\sum_{i=1}^{k} \boldsymbol{R}_{ii} \tag{20}$$

where the classes number is represented by k, the samples number is represented by Q, and the confusion matrix is denoted as $\boldsymbol{R} \in \mathbb{R}^{k\times k}$.

Next, we will discuss parameters settings that have a significant impact on the classification performance of the DHTCSCNet, which includes the patch number p, the patches size d, and the number of network layers l.

[1]http://www.ehu.eus/ccwintco/index.php/Hyperspectral_Remote_Sensing_Scenes#Pavia_University_scene

[2]http://www.ehu.eus/ccwintco/index.php/Hyperspectral_Remote_Sensing_Scenes

TABLE II
STATISTICS FOR THE PAVIAU

Class number	Class name	Training samples	Test samples
1	Asphalt	190	6441
2	Meadows	190	18459
3	Gravel	160	1939
4	Trees	170	2894
5	Metal sheets	170	1175
6	Bare soil	170	4859
7	Bitumen	160	1170
8	Bricks	165	3517
9	Shadows	165	782
	Total	1540	41236

TABLE III
STATISTICS FOR THE IP

Class number	Class name	Training samples	Test samples
1	Alfalfa	30	16
2	Corn-notill	150	1278
3	Corn-mintill	150	680
4	Corn	100	137
5	Grass-pasture	150	333
6	Grass-trees	150	580
7	Grass-pasture-mowed	20	8
8	Hay-windrowed	150	328
9	Oats	15	5
10	Soybean-notill	150	822
11	Soybean-mintill	150	2305
12	Soybean-clean	150	443
13	Wheat	150	55
14	Woods	150	1115
15	Buildings-Grass-Trees-Drives	50	336
16	Stone-Steel-Towers	50	43
	Total	1765	8484

TABLE IV
STATISTICS FOR THE KSC

Class number	Class name	Training samples	Test samples
1	Scrub	33	728
2	Willow swamp	23	220
3	CP hammock	24	232
4	CP/Oak	24	228
5	Slash pine	15	146
6	Oak/Broadleaf	22	207
7	Hardwood swamp	9	96
8	Graminoid marsh	38	393
9	Spartina marsh	51	469
10	Catiail marsh	39	365
11	Salt marsh	41	378
12	Mud flats	49	454
13	Water	91	836
	Total	459	4752

a) Effects of number of patches: In order to study whether the classification performance is affected by p, we set p to take a value from 2 to 34 with a gap of 2, and the results are shown on the left-hand side of Fig. 3. As shown on the left-hand side of Fig. 3, as the number of image patches increases, classification accuracy first increases and then decreases, which indicates that the classification performance is not always positively correlated with the number of image patches. When p ranges from 2 to 24 on the KSC and IP datasets, classification accuracy increases but decreases on the PaviaU dataset when p ranges from 2 to 26. As shown in Fig. 3, when $p = 24$, the proposed method on KSC and IP datasets achieves the highest classification accuracy; when $p = 26$, the proposed method on the PaviaU dataset achieves the highest classification accuracy. Choosing $p = 26$ does not make the proposed method achieve the highest classification accuracy on the three datasets. As a tradeoff between $p = 24$ and $p = 26$, we choose $p = 24$ with a lower computational cost.

b) Effects of patch size: To analyze the impact of different patch size d on classification accuracy, we set d to take a value from 3 to 27 with a gap of 2. As shown in the middle of Fig. 3, the size of patches affects the classification performance.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:55:14 UTC from IEEE Xplore. Restrictions apply.

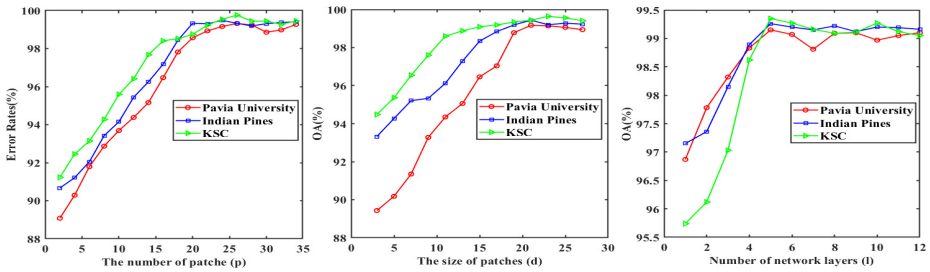


Fig. 3. Influence of different parameters on DHTCSCNet.

TABLE V
QUANTITATIVE CLASSIFICATION ACCURACIES ON THE PAVIAU

Class	GTR	SAE	EMP	LeNet	RPNet	GSL	SSAN	SBNN	MGCN	MFDN	S^2GSAGE	SSRPNet	Ours
1	99.04	92.25	97.72	98.32	95.48	97.93	98.68	97.17	85.70	98.34	84.87	99.78	**99.83**
2	98.64	91.19	97.08	97.05	97.42	98.37	99.44	99.12	92.94	99.93	95.29	99.51	**99.99**
3	98.98	89.39	98.64	98.00	98.43	85.99	86.00	85.09	92.11	92.55	85.96	**99.75**	99.38
4	93.73	98.87	98.98	**99.00**	98.60	92.72	98.33	95.19	83.98	94.63	83.63	92.21	90.74
5	99.36	99.97	99.94	99.73	**100**	99.18	99.92	99.19	96.53	99.90	99.52	**100**	**100**
6	99.89	93.28	96.59	97.51	98.59	93.82	99.11	94.05	98.65	99.60	85.85	**100**	**100**
7	99.79	96.36	98.24	99.31	98.50	80.75	96.55	87.36	96.61	99.71	96.78	99.84	**100**
8	99.11	96.39	98.95	98.05	99.05	96.90	94.07	95.62	88.56	97.30	94.16	**99.86**	98.52
9	98.53	99.80	99.21	99.88	99.89	97.68	**100**	96.50	90.19	95.19	97.67	94.80	97.19
OA	98.58	92.97	97.61	97.73	97.70	96.09	98.02	96.53	91.60	98.89	91.41	99.05	**99.11**
Kappa× 100	98.13	90.55	96.75	96.91	96.96	94.80	97.37	95.39	89.00	98.10	88.61	98.73	**98.81**

TABLE VI
QUANTITATIVE CLASSIFICATION ACCURACIES ON THE IP

Class	GTR	SAE	EMP	LeNet	RPNet	GSL	SSAN	SBNN	MGCN	MFDN	S^2GSAGE	SSRPNet	Ours
1	92.35	**100**	94.79	**100**	**100**	**100**	80.49	51.82	**100**	97.20	**100**	**100**	**100**
2	96.48	82.86	89.73	92.78	94.02	97.86	90.82	93.84	78.74	95.16	74.21	**99.69**	**99.69**
3	**99.92**	94.10	95.37	97.98	96.59	95.56	93.84	96.09	94.12	96.57	88.28	99.12	99.41
4	91.49	92.63	94.25	99.48	99.20	98.51	89.20	92.62	98.55	90.48	96.18	**100**	**100**
5	96.14	97.15	97.92	98.31	99.00	98.52	99.08	94.03	90.18	91.99	94.45	99.70	**100**
6	98.87	98.60	99.10	98.87	99.69	**100**	99.24	97.89	94.57	98.93	98.39	**100**	**100**
7	96.67	98.75	99.58	**100**	91.67	**100**	96.00	73.33	**100**	90.90	**100**	**100**	**100**
8	99.78	99.79	**100**	99.74	99.91	**100**	98.14	99.78	**100**	99.75	99.69	**100**	**100**
9	**100**	**100**	**100**	**100**	**100**	**100**	82.90	**100**	67.37	**100**	**100**	**100**	**100**
10	96.26	90.29	93.10	94.35	96.36	96.84	94.62	93.28	85.67	95.35	90.18	98.54	**100**
11	98.58	84.39	84.61	93.45	93.08	96.84	98.10	97.59	90.37	96.52	80.32	98.57	**99.09**
12	96.28	93.93	95.31	98.26	97.72	97.53	94.65	89.77	90.90	90.04	93.49	99.32	**99.77**
13	99.40	99.64	99.63	**100**	99.70	93.22	**100**	97.23	**100**	98.76	98.12	**100**	**100**
14	**100**	96.78	98.66	99.05	99.21	99.55	98.42	97.55	99.76	98.37	94.35	**100**	99.91
15	96.84	85.51	91.99	98.13	95.76	99.67	82.71	94.12	98.67	96.53	88.07	**100**	**100**
16	96.33	99.77	95.34	99.00	98.37	97.73	91.57	99.32	98.41	99.01	98.61	**100**	**100**
OA	97.92	90.11	92.22	96.01	96.09	97.09	95.49	95.47	91.24	96.08	87.43	99.30	**99.39**
Kappa× 100	97.63	85.56	91.12	95.36	95.46	96.61	94.85	94.83	90.00	95.26	85.66	99.19	**99.29**

As shown in Fig. 3, as d increases, classification accuracy on PaviaU, KSC, and IP datasets tends to increase and then decrease slightly. A larger patch size cannot achieve the highest classification accuracy. For all three datasets, when $d = 21$, the classification accuracy of the proposed method on the three datasets is not the highest. The larger the image patch, the greater the computational cost. Here, we set d to 21.

c) Effect of the number of network layers: To analyze the impact of different numbers of network layers l on classification accuracy, we set l from 1 to 12 for all three datasets. As shown on the right-hand side of Fig. 3, as l increases from 1 to 5, the values of OAs on the PaviaU, KSC, and IP datasets also increase. When l increases from 5 to 12, the OAs on the PaviaU, KSC, and IP have a tendency to decrease. Therefore, choosing $l = 5$ for all three datasets is a better choice.

B. Classification Results

In order to evaluate the classification performance of DHTCSCNet, we introduce some closely related hyperspectral classification methods for comparison, including generalized tensor regression (GTR) [25], stacked autoencoder (SAE) [35], extended morphological profile (EMP) [36], Lecun *et al.* [37] proposed CNN (LeNet), random patches network (RPNet) [4], and GSL [38]. In addition, some the state-of-the-art deep learning methods, such as SSAN [39], SBNN [40], MGCN [41], MFDN [42], S^2GSAGE [43], and SSRPNet [2], are also compared with the proposed method on the three datasets. Tables V–VII show the quantitative assessment. The classification maps of different methods are shown in Figs. 4–6. Here, the results of the comparison methods reported in our article are all derived from the results in their corresponding articles.

1) Pavia University Dataset: As shown in Table V, the OA and kappa coefficient of the proposed method are the highest among all compared methods. Compared with RPNet and LeNet, OA of the DHTCSCNet is more than 1.3%; OA of the DHTCSCNet is more than 3% compared with GSL and more than 0.5% compared with GTR, while kappa coefficient of the proposed method is more than 1.8% compared with RPNet and LeNet, more than 4% compared with GSL, and more than

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:55:14 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
QUANTITATIVE CLASSIFICATION ACCURACIES ON THE KSC

Class	GTR	SAE	EMP	LeNet	RPNet	GSL	SBNN	MGCN	MFDN	S^2GSAGE	SSRPNet	Ours
1	97.94	91.61	95.74	91.74	96.95	96.69	99.81	98.42	**99.84**	99.70	96.70	**100**
2	92.73	83.74	95.65	90.09	96.79	95.83	92.12	**99.60**	96.81	98.99	98.64	93.64
3	94.83	76.20	96.50	86.34	98.32	94.04	96.87	98.52	97.11	99.37	**100**	**100**
4	84.21	59.99	93.75	76.80	94.75	95.07	83.85	97.79	86.33	78.26	**100**	**100**
5	84.93	79.15	85.11	92.40	91.21	93.38	76.08	98.40	86.32	83.05	**100**	**100**
6	95.65	68.44	91.15	90.34	95.41	93.33	91.38	98.54	93.32	80.80	**100**	**100**
7	88.54	92.82	97.67	90.94	98.47	97.96	98.80	98.53	99.51	99.44	**100**	**100**
8	98.73	89.77	97.98	94.35	98.15	94.62	99.76	99.00	99.72	98.53	**100**	**100**
9	**100**	92.25	99.70	97.85	99.39	93.89	99.39	99.45	97.35	**100**	**100**	**100**
10	93.15	96.10	97.64	99.48	99.18	95.86	98.54	99.92	95.78	96.56	**100**	**100**
11	95.24	99.71	**100**	99.89	99.71	97.26	99.30	**100**	99.96	99.64	**100**	**100**
12	81.06	98.78	97.76	98.55	99.72	94.12	98.20	99.90	98.60	94.83	**100**	**100**
13	**100**	**100**	**100**	**100**	**100**	98.70	**100**	**100**	**100**	99.43	**100**	**100**
OA	94.63	90.67	97.19	94.79	98.22	95.85	97.10	99.29	97.55	96.67	99.43	**99.60**
Kappa× 100	94.03	89.60	96.87	94.20	98.02	95.38	96.77	99.21	97.27	96.27	99.37	**99.56**

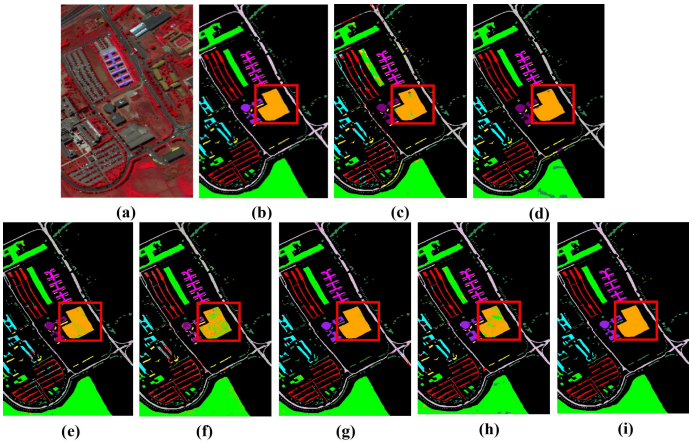


Fig. 4. Visualization of classification results on the PaviaU dataset. (a) False color maps. (b) Real feature marker. (c) GTR [25]. (d) SAE [35]. (e) Extended morphological profile [36]. (f) Lecun [37]. (g) RPNet [4]. (h) GSL [38]. (i) DHTCSCNet.

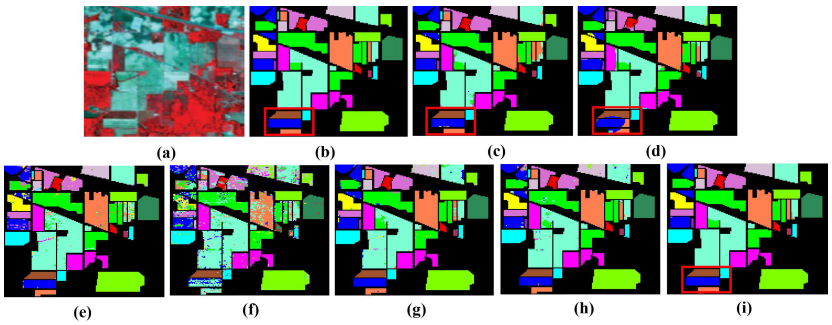


Fig. 5. Visualization of classification results on the IP dataset. (a) False color maps. (b) Real feature marker. (c) GTR [25]. (d) SAE [35]. (e) Extended morphological profile [36]. (f) Lecun [37]. (g) RPNet [4]. (h) GSL [38]. (i) DHTCSCNet.

0.6% compared with GTR. Fig. 4 shows the visualized results of the DHTCSCNet on PaviaU, and the DHTCSCNet produces classification maps with less noise. We can see from Fig. 4 that classification maps obtained by GTR, EMP, LeNet, and GSL produce more noise, as shown in the red box in Fig. 4. For the visualized results of GTR, SAE, RPNet, and DHTCSCNet in Fig. 4 are most similar to the real feature marker (ground-truth maps), but GTR, SAE, and RPNet are still misclassified, as shown in the red box in Fig. 4. In addition, some of the state-of-the-art deep learning methods [39]–[42], and [43] are also compared with the proposed method; OA and the kappa coefficient of the proposed method are significantly higher than compared methods.

2) Indian Pines Dataset: Table VI shows the quantitative results of different methods. It can be seen from the table that OA and the kappa coefficient of the proposed method are the highest. Compared with RPNet and LeNet, the OA of DHTCSCNet is more than 3.3%, more than 2.3% compared with GSL, and more than 1.4% compared with GTR, while the kappa coefficient of the proposed method is more than 1.6% compared with GTR, more than 3.8% compared with RPNet and LeNet, and more than 2.6% compared with GSL. Fig. 5 shows the classification maps of the proposed method on Indian Pines dataset. As shown in Fig. 5, the DHTCSCNet shows excellent performance. The classification maps generated by [4], [36], [37] and [38] produce more noise.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:55:14 UTC from IEEE Xplore. Restrictions apply.

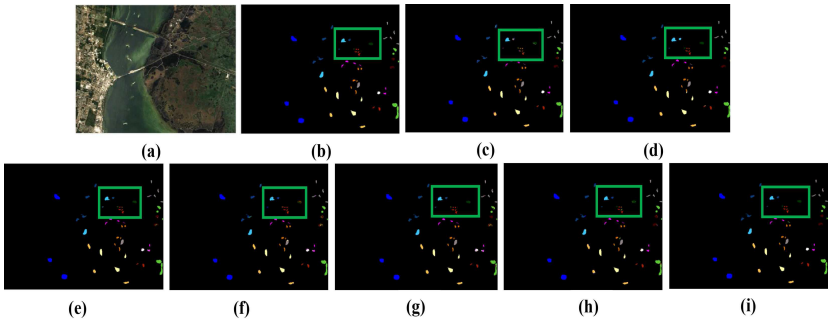


Fig. 6. Visualization of classification results on the KSC dataset. (a) False color maps. (b) Real feature marker. (c) GTR [25]. (d) SAE [35]. (e) Extended morphological profile [36]. (f) Lecun [37]. (g) RPNet [4]. (h) GSL [38]. (i) DHTCSCNet.

The visualized results of GTR, SAE, and DHTCSCNet in Fig. 5 are most similar to the real feature marker, but GTR and SAE are still misclassified, as shown in the red marked area. Compared with some state-of-the-art deep learning methods [39]–[42], and [43], OA and the kappa coefficient of the proposed method are more than 3.3% and 4%, respectively.

3) KSC Dataset: The quantitative evaluation results of the different methods are presented in Table VII. We can see from Table VII that the DHTCSCNet obtains the best classification performance among all comparison methods. The OA of DHTCSCNet is more than 4.9% compared with GTR, more than 1.3% compared with RPNet, more than 4.8% compared with LeNet, and more than 3.7% compared with GSL, while the kappa coefficient of DHTCSCNet is more than 5.5% compared with GTR, more than 1.5% compared with RPNet, more than 5.3% compared with LeNet, and more than 4.1% compared with GSL. From Fig. 6, the visualized results obtained by GTR, SAE, EMP, LeNet, RPNet, and GSL are misclassified, as shown in the green box in Fig. 6. The visualized results of SAE and DHTCSCNet in Fig. 6 are most similar to the ground-truth maps, but SAE is still misclassified, as shown in the green marked area. In addition, some state-of-the-art deep learning methods [39]–[42], and [43] are compared with the proposed method, and the proposed method achieves the best classification performance among all comparison methods. OA and the kappa coefficient of the DHTCSCNet are more than 2% compared with SSAN, SBNN, MFDN, and S^2GSAGE, and more than 0.3% compared with MGCN.

C. Analysis and Discussion

In order to further investigate the effectiveness of the DHTCSCNet, we conduct the ablation study. In the ablation study experiment, all random filters in RandomNet are generated randomly, all filters in DHTCSCNet are generated by the deep high-order tensor CSC model, and the network structure of both remains the same.

1) Ablation Study: We can see from Table VIII that the DHTCSCNet achieves high classification accuracy on the PaviaU, IP, and KSC datasets. On the PaviaU dataset, OA and the kappa coefficient of the DHTCSCNet surpass RandomNet by 0.4%. On the IP dataset, the OA of the DHTCSCNet surpasses RandomNet by 3.5%, and the kappa coefficient of the DHTCSCNet surpasses RandomNet by 4.1%. On the KSC dataset, the OA of the DHTCSCNet surpasses RandomNet by 1.5%, and the kappa coefficient of the DHTCSCNet surpasses RandomNet by 1.6%. This indicates that the DHTCSCNet using filters generated by the deep high-order tensor CSC model can achieve better classification performance than the RandomNet using filters generated randomly.

TABLE VIII

QUANTITATIVE RESULTS OF THE DHTCSCNET USING DIFFERENT INITIALIZATION METHODS

Datasets	Metric	DHTCSCNet	RandomNet
Pavia University	OA	**99.11**	98.71
	Kappa× 100	**98.81**	98.24
Indian Pines	OA	**99.39**	95.80
	Kappa× 100	**99.29**	95.12
KSC	OA	**99.60**	98.08
	Kappa× 100	**99.56**	97.87

TABLE IX

EFFECT OF TRAINING SAMPLES NUMBER ON THREE DATASETS

	Training Samples	1%	3%	5%	7%	10%	15%	20%
Pavia University	GRT	89.47	92.22	92.21	92.39	92.14	92.91	92.59
	RPNet	89.78	96.63	97.95	99.03	99.32	99.62	99.74
	GSL	71.51	90.93	96.84	97.79	98.96	99.03	99.25
	LeNet	84.58	87.32	89.66	90.14	92.67	93.14	95.38
	DHTCSCNet	**90.70**	**97.29**	**99.17**	**99.32**	**99.58**	**99.71**	**99.90**
	Training Samples	3%	5%	7%	10%	12%	15%	20%
Indian Pines	GRT	88.27	95.37	96.21	97.17	97.74	98.36	98.33
	RPNet	86.31	91.06	93.22	96.16	96.89	97.22	98.21
	GSL	85.50	95.22	97.18	98.23	98.66	98.93	99.23
	LeNet	64.69	69.58	74.08	80.26	82.67	89.17	91.53
	DHTCSCNet	**93.62**	**96.68**	**97.94**	**98.64**	**98.99**	**99.35**	**99.60**
	Training Samples	3%	5%	7%	10%	12%	15%	20%
KSC	GRT	85.89	92.41	95.05	97.31	94.18	95.03	97.46
	RPNet	86.57	93.10	93.45	94.77	95.68	96.24	96.61
	GSL	82.33	95.02	97.52	98.61	99.61	99.76	99.83
	LeNet	82.55	85.58	87.11	90.21	90.42	92.05	93.45
	DHTCSCNet	**95.30**	**98.76**	**99.27**	**99.55**	**100**	**100**	**100**

2) Impact of Training Ratio: Then, an experiment was conducted on whether the classification performance is affected by the number of training samples, and the experimental results are shown in Table IX. For the PaviaU, KSC, and IP datasets, $\imath\%$ (the value of $\imath$ is shown in Table IX) of the total sample number was selected from each class with labeled data as the training samples and the remaining data as the test sample. The results in Table IX that, when the number of training samples increases, the classification performances of the DHTCSCNet, GRT, RPNet, GSL, and LeNet also improve. Among these methods, the classification accuracy of the DHTCSCNet is always higher than those of others

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:55:14 UTC from IEEE Xplore. Restrictions apply.

TABLE X
TIME COSTS FOR THE PAVIAU, IP, AND KSC

Dataset	GTR	SAE	EMP	LeNet	RPNet	GSL	SSAN	SBNN	MGCN	MFDN	S^2GSAGE	SSRPNet	DHTCSCNet
Pavia University	69.02s	374.23s	518.73s	432.45s	44.81s	325.82s	1278.04s	76.25s	244s	803.59s	1900s	38.42s	**31.75**s
Indian Pines	38.34s	242.70s	136.69s	68.23s	14.96s	53.01s	1787.24s	88.01s	95s	726.40s	2485s	16.83s	**13.29**s
KSC	53.73s	178.12s	425.29s	96.78s	34.51s	15.26s	-	-	51s	549.60s	1709s	13.07s	**11.83**s

TABLE XI
COMPARISON OF COMPUTATIONAL COMPLEXITY (FLOPS) OF DIFFERENT METHODS

Methods	LeNet	SSAN	SBNN	MFDN	SSRPNet	RPNet	GTR	Ours
Complexity	4.6×10^7	1.2×10^9	6.7×10^7	5.3×10^8	2.1×10^7	2.9×10^7	3.3×10^7	1.5×10^7

methods. When $\iota\%$ is greater than or equal to 12%, the classification accuracy (OA) of the DHTCSCNet on the KSC dataset reaches 100%. The results in Table IX show that, when the number of training samples reaches 7% of the total number of samples, the OAs of the DHTCSCNet on the three datasets achieve 99.32%, 97.94%, and 99.27%, respectively. This shows the effectiveness of the DHTCSCNet in the case of small samples.

Table X displays time costs of different methods, including the deep learning methods SAE [35], Lecun *et al.* [37] proposed CNN (LeNet), RPNet [4], SSAN [39], SBNN [40], MGCN [41], MFDN [42], and S^2GSAGE [43]. Because deep learning methods usually require pretraining and fine-tuning, they often take longer than our method (without fine-tuning process). As shown in Table X, [4], [35], [37], [39], [40]–[42], and [43] are slower than the proposed method. Compared with the traditional methods GTR and extended morphological profile, the proposed DHTCSCNet takes shorter. The time of the DHTCSCNet is shorter than that of the RPNet without pretraining. Compared with GTR, the DHTCSCNet is about two times faster, ten times faster compared with extended morphological profile, and 1.2 times faster compared with RPNet.

Table XI shows the computational complexity (floating-point operations, abbreviated as flops) of different methods, including LeNet [37], RPNet [4], SSAN [39], SBNN [40], GTR [25], MFDN [42], and SSRPNet [2]. From the results in the table, the computational complexity of our method is the lowest. The computational complexity of SSRPNet is 1.4 times that of our method. The computational complexity of RPNet and GTR is about two times that of our method. The computational complexity of MFDN is about 35 times that of our method. The computational complexity of LeNet and SBNN is about three times that of our method. The computational complexity of SSAN is 80 times that of our method.

V. CONCLUSION

In this article, we propose a deep high-order tensor CSC model, which can be used to train deep high-order filters. Based on the deep high-order tensor CSC model, a DHTCSCNet is constructed, which can be used to extract deep features. Combining the spectral–spatial feature and the features extracted by the proposed DHTCSCNet at each layer, the learning model based on the graph is used for classification. The proposed DHTCSCNet obtains better classification performance compared with other HSI classification methods.

In the future, we will continue to study efficient solvers for deep higher-order tensor CSC models and design an efficient feature extractor.

REFERENCES

[1] K. Gao, W. Guo, X. Yu, B. Liu, A. Yu, and X. Wei, "Deep induction network for small samples classification of hyperspectral images," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 3462–3477, 2020.

[2] C. Cheng, H. Li, J. Peng, W. Cui, and L. Zhang, "Hyperspectral image classification via spectral–spatial random patches network," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 4753–4764, 2021.

[3] X. Chen, S. Xiang, C.-L. Liu, and C.-H. Pan, "Vehicle detection in satellite images by hybrid deep convolutional neural networks," *IEEE Geosci. Remote Sens. Lett.*, vol. 11, no. 10, pp. 1797–1801, Oct. 2014.

[4] Y. Xu, B. Du, F. Zhang, and L. Zhang, "Hyperspectral image classification via a random patches network," *ISPRS J. Photogramm. Remote Sens.*, vol. 142, pp. 344–357, Aug. 2018.

[5] J. C. Harsanyi and C.-I. Chang, "Hyperspectral image classification and dimensionality reduction: An orthogonal subspace projection approach," *IEEE Trans. Geosci. Remote Sens.*, vol. 32, no. 4, pp. 779–785, Jul. 1994.

[6] J. Bai, S. Xiang, L. Shi, and C. Pan, "Semisupervised pair-wise band selection for hyperspectral images," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 8, no. 6, pp. 2798–2813, Jul. 2015.

[7] S. Rajan, J. Ghosh, and M. M. Crawford, "An active learning approach to hyperspectral data classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 46, no. 4, pp. 1231–1242, Apr. 2008.

[8] J. Peng, L. Li, and Y. Y. Tang, "Maximum likelihood estimation-based joint sparse representation for the classification of hyperspectral remote sensing images," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 6, pp. 1790–1802, Jun. 2019.

[9] J. Li, J. M. Bioucas-Dias, and A. Plaza, "Spectral–spatial classification of hyperspectral data using loopy belief propagation and active learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 2, pp. 844–856, Feb. 2013.

[10] J. Yue, W. Zhao, S. Mao, and H. Liu, "Spectral–spatial classification of hyperspectral images using deep convolutional neural networks," *Remote Sens. Lett.*, vol. 6, no. 6, pp. 468–477, May 2015.

[11] J. Peng, W. Sun, and Q. Du, "Self-paced joint sparse representation for the classification of hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 1183–1194, Feb. 2019.

[12] L. Shen and S. Jia, "Three-dimensional Gabor wavelets for pixel-based hyperspectral imagery classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 49, no. 12, pp. 5039–5046, Dec. 2011.

[13] Y. Tarabalka, M. Fauvel, J. Chanussot, and J. A. Benediktsson, "SVM- and MRF-based method for accurate classification of hyperspectral images," *IEEE Geosci. Remote Sens. Lett.*, vol. 7, no. 4, pp. 736–740, Oct. 2010.

[14] S. Jia, J. Hu, J. Zhu, X. Jia, and Q. Li, "Three-dimensional local binary patterns for hyperspectral imagery classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 4, pp. 2399–2413, Apr. 2017.

[15] F. Gao, Q. Wang, J. Dong, and Q. Xu, "Spectral and spatial classification of hyperspectral images based on random multi-graphs," *Remote. Sens.*, vol. 10, no. 8, pp. 1271–1290, 2018.

[16] J. Liu, Z. Wu, Z. Wei, L. Xiao, and L. Sun, "Spatial–spectral kernel sparse representation for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 6, no. 6, pp. 2462–2471, Dec. 2013.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:55:14 UTC from IEEE Xplore. Restrictions apply.

[17] G. Camps-Valls, L. Gomez-Chova, J. Munoz-Mari, J. Vila-Frances, and J. Calpe-Maravilla, "Composite kernels for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 3, no. 1, pp. 93–97, Jan. 2006.

[18] H. Li, Y. Song, and C. L. Philip Chen, "Hyperspectral image classification based on multiscale spatial information fusion," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 9, pp. 5302–5312, Sep. 2017.

[19] Z. Ye, H. Li, Y. Song, J. A. Benediktsson, and Y. Y. Tang, "Hyperspectral image classification using principal components-based smooth ordering and multiple 1-D interpolation," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 2, pp. 1199–1209, Feb. 2017.

[20] J. Chen, W. Zhang, Y. Qian, and M. Ye, "Deep tensor factorization for hyperspectral image classification," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2018, pp. 4788–4791.

[21] J. Peng *et al.*, "Low-rank and sparse representation for hyperspectral image processing: A review," *IEEE Geosci. Remote Sens. Mag.*, early access, Jun. 10, 2021, doi: 10.1109/MGRS.2021.3075491.

[22] D. Lee and H. Seung, "Algorithms for non-negative matrix factorization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2001, pp. 556–562.

[23] C. Cheng, H. Li, and L. Zhang, "Two-branch convolutional sparse representation for stereo matching," *IEEE Access*, vol. 9, pp. 21910–21920, 2021.

[24] X. Guo, X. Huang, L. Zhang, L. Zhang, A. Plaza, and J. A. Benediktsson, "Support tensor machines for classification of hyperspectral remote sensing imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 6, pp. 3248–3264, Jun. 2016.

[25] J. Liu, Z. Wu, L. Xiao, J. Sun, and H. Yan, "Generalized tensor regression for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 2, pp. 1244–1258, Feb. 2020.

[26] Y. Qian, F. Xiong, S. Zeng, J. Zhou, and Y. Y. Tang, "Matrix-vector nonnegative tensor factorization for blind unmixing of hyperspectral imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 3, pp. 1776–1792, Mar. 2017.

[27] A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, vol. 25, no. 2, pp. 1097–1105.

[28] S. Ji, W. Xu, M. Yang, and K. Yu, "3D convolutional neural networks for human action recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 1, pp. 221–231, Jan. 2013.

[29] Y. Sun, X. Wang, and X. Tang, "Deep learning face representation from predicting 10,000 classes," in *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, Jun. 2014, pp. 1891–1898.

[30] M. Xi, L. Chen, D. Polajnar, and W. Tong, "Local binary pattern network: A deep learning approach for face recognition," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Sep. 2016, p. 3224.

[31] X. Ren, K. Chen, X. Yang, Y. Zhou, J. He, and J. Sun, "A new unsupervised convolutional neural network model for Chinese scene text detection," in *Proc. IEEE China Summit Int. Conf. Signal Inf. Process.*, Jul. 2015, pp. 428–432.

[32] J. Li, T. Zhang, W. Luo, J. Yang, X.-T. Yuan, and J. Zhang, "Sparseness analysis in the pretraining of deep neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 6, pp. 1425–1438, Jun. 2017.

[33] A. Bibi and B. Ghanem, "High order tensor formulation for convolutional sparse coding," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 1790–1798.

[34] W. Huang, Y. Huang, Z. Wu, J. Yin, and Q. Chen, "A multi-kernel mode using a local binary pattern and random patch convolution for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 4607–4620, 2021.

[35] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.

[36] J. A. Benediktsson, J. A. Palmason, and J. R. Sveinsson, "Classification of hyperspectral data from urban areas based on extended morphological profiles," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 3, pp. 480–491, Mar. 2005.

[37] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[38] L. Ma, A. Ma, C. Ju, and X. Li, "Graph-based semi-supervised learning for spectral–spatial hyperspectral image classification," *Pattern Recognit. Lett.*, vol. 83, pp. 133–142, Nov. 2016.

[39] H. Sun, X. Zheng, X. Lu, and S. Wu, "Spectral–spatial attention network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3232–3245, May 2020.

[40] M. E. Paoletti, J. M. Haut, X. Tao, J. Plaza, and A. Plaza, "FLOP-reduction through memory allocations within CNN for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5938–5952, Jul. 2021, doi: 10.1109/TGRS.2020.3024730.

[41] S. Wan, C. Gong, P. Zhong, B. Du, L. Zhang, and J. Yang, "Multiscale dynamic graph convolutional network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3162–3177, Nov. 2019.

[42] Z. Li *et al.*, "Deep multilayer fusion dense network for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 1258–1270, 2020.

[43] P. Yang, L. Tong, B. Qian, Z. Gao, J. Yu, and C. Xiao, "Hyperspectral image classification with spectral and spatial graph using inductive representation learning network," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 791–800, 2021.



Chunbo Cheng received the B.Sc. degree in mathematics and applied mathematics from Yangtze University, Jingzhou, China, in 2015, and the Ph.D. degree from the School of Mathematics and Statistics, Huazhong University of Science and Technology, Wuhan, China, in 2020.

He is currently a Lecturer with the College of Mathematical and Physical Sciences, Hubei Polytechnic University, Huangshi, China. His research interests include pattern recognition, machine learning, and computer vision.



Hong Li (Member, IEEE) received the M.Sc. degree in mathematics and the Ph.D. degree in pattern recognition and intelligence control from the Huazhong University of Science and Technology, Wuhan, China, in 1986 and 1999, respectively.

She is currently a Professor with the School of Mathematics and Statistics, Huazhong University of Science and Technology. Her research interests include approximation theory, wavelet analysis, learning theory, neural networks, signal processing, and pattern recognition.



Jiangtao Peng (Senior Member, IEEE) received the B.S. and M.S. degrees from Hubei University, Wuhan, China, in 2005 and 2008, respectively, and the Ph.D. degree from the Institute of Automation, Chinese Academy of Sciences, Beijing, China, in 2011.

He is currently a Professor with the Faculty of Mathematics and Statistics, Hubei University. His research interests include machine learning and hyperspectral image processing.



Wenjing Cui received the B.S. degree from Henan Normal University, Henan, China, in 2015, and the M.S. degree from the Wuhan University of Technology, Wuhan, China, in 2018.

She is currently an Assistant with the College of Mathematical and Physical Sciences, Hubei Polytechnic University, Huangshi, China. Her research interests include machine learning and econometric analysis.



Liming Zhang (Member, IEEE) received the M.S. degree in signal processing from the Nanjing University of Science and Technology, Nanjing, China, and the Ph.D. degree in image processing from the University of New England, Armidale, NSW, Australia.

She is currently an Assistant Professor with the Faculty of Science and Technology, University of Macau, Macau, China. Her research interests include signal processing, image processing, and computer vision.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:55:14 UTC from IEEE Xplore. Restrictions apply.